Interview Prep — The Live Agent

> *Study material for the pms-semantic-layer project. Continues 02-agent-routing-and-scope.md; continues in 04-senior-review-walkthrough.md.*

Continues the format and numbering from 02-agent-routing-and-scope.md (sections 1-14). This file covers building the actual agent that loads `agent_routing.yml` / `agent_scope.yml` and answers real questions.

---

15. Enforcing scope at the database layer vs. the application layer

Q: `agent_scope.yml` declares `allowed_schemas` / `read_only`. Why enforce that with a dedicated Postgres role instead of just checking the generated SQL text in Python before running it?

- **Weak answer**: "Checking the SQL text is enough if the check is thorough."
- **The gap**: an application-level text check has to anticipate every way a forbidden reference could be phrased -- exactly the trap chapter 07's `is_read_only` already hit once (SQL comments hiding the real statement) and chapter 11 hit again in a different form (a generator whose invariant only held by coincidence). Every text-based check is a denylist/allowlist built by a human predicting LLM output, and LLM output is the one thing in this pipeline that isn't fully predictable.
- **Correct answer**: create a Postgres role (`pms_agent`) with `GRANT SELECT` on `public_marts` only, and nothing else -- `REVOKE ALL` on `public` / `public_staging`. Now an attempt to read staging or write anywhere fails at the database engine itself, regardless of how the SQL was phrased, what the LLM "meant," or whether the application-level check has a bug. This is the same principle as chapter 09's `CHECK (booked_at <= start_date)` constraint: don't trust application code to catch what the database can guarantee structurally.
- **Study**: this is the general "defense should live at the layer that can't be bypassed" principle -- a role's privileges are checked by Postgres on every single statement, with no code path around it, unlike an app-level check that only runs if someone remembers to call it.

Q: If the database role already blocks writes (no INSERT/UPDATE/DELETE granted), is chapter 07's `is_read_only` check still worth keeping?

- **Weak answer**: "No, it's redundant now."
- **The gap**: `is_read_only` and a read-only DB role protect against two different things. The DB role stops privilege violations (writing, reading a forbidden schema). It does nothing about a syntactically-read-only but still undesirable statement -- e.g. a multi-statement payload (`SELECT 1; SELECT pg_sleep(1000)`) that's technically all `SELECT` s and would execute happily under a read-only role, but is still a shape of query the agent should never construct or run.
- **Correct answer**: keep both. The DB role is the hard, unbypassable backstop for privilege; `is_read_only` is an earlier, cheaper check for statement *shape* (single statement, starts with `SELECT` / `WITH`) that the database's permission system was never designed to catch. Defense-in-depth means each layer catches what the other structurally can't, not that one makes the other pointless.

16. Reusing `fetchmany` as the real enforcement for `max_rows`

Q: `agent_scope.yml` declares `max_rows: 500`. Where does that number actually get enforced?

- **Weak answer**: "The agent checks that the generated SQL includes `LIMIT 500` ."
- **The gap**: same failure mode as trusting SQL text for schema/column checks -- an LLM might omit the `LIMIT` , put the wrong number, or the query might not need a `LIMIT` clause to still return

more rows than intended (e.g. a `GROUP BY` with no filter). Relying on the *generated text* to self-limit is fragile for the same reason relying on it to self-restrict schemas is fragile.

- **Correct answer**: enforce it the same way chapter 07 enforced its own result-size cap -- `cur.fetchmany(500)` at the Python/psycopg2 layer, independent of whatever the SQL itself says. The database can return however many rows match; the application simply never pulls more than 500 of them into memory. This is the exact same mechanism as `run_demo.py` 's `fetchmany(6)` , scaled up for a real analytics agent instead of a demo transcript preview -- the number changed, the enforcement point (execution, not generation) didn't.

17. Good context reduces hallucination, it doesn't remove the need for validation

Q: Now that the agent gets a real, scoped schema context (unlike chapter 07's zero context), can the validation layer (`is_read_only` , the `pms_agent` DB role) be relaxed?

- **Weak answer**: "Mostly, since the model now knows the real schema."
- **The gap**: context fixes a specific failure mode -- guessing a table/column that doesn't exist. It does nothing about a different failure mode -- generating a query that references the *right* tables/columns but combines them wrong (a bad join key, ignoring a documented caveat in `notes`). A well-grounded query can still be confidently, plausibly wrong.
- **Correct answer**: keep every validation layer regardless of how good the context is -- they defend against different things. Good context lowers the *rate* of certain mistakes; it does not change what happens when a mistake still occurs. A second, smaller risk worth naming: if `match_question_type` matches the wrong `question_type` , the agent is now scoped to tables that don't actually answer the question -- a different failure shape than chapter 07's, but still a failure, and nothing about "the context looks good" would surface it on its own.

18. Two real bugs found by actually running the agent, not by inspection

Q: `agent_routing.yml` and `agent_scope.yml` were both reviewed carefully before this chapter. Why did running the real agent against real questions still surface new bugs?

- **Weak answer**: "The config wasn't reviewed thoroughly enough."
- **The gap**: no amount of static review catches everything an actual LLM will do with a prompt -- config review checks the *design*, running the agent checks the *behavior*, and those diverge in ways that are hard to predict from reading YAML alone.
- **Bug 1 -- a tie-break regression introduced by a design improvement.** Switching `match_question_type` from substring matching to word-set matching (chapter 13, to fix a real miss where "normalized daily cost" didn't match "normalized cost") made word-count ties far more likely, since many keywords across different types share the same word count. A real question ("which month had the most cancelled reservations...") tied between `cancelled_reservations_list` 's `"cancelled reservations"` (2 words) and `cancellations_and_lost_revenue_by_month` 's `"lost revenue"` (2 words) -- the tie silently resolved to whichever type happened to appear first in the YAML, not the semantically correct one. The wrong type's context lacked the note specifying "group by start_date, not cancelled_at" -- and the model, left to decide on its own, picked a different (still defensible, but different) business definition. **Fix**: not a smarter generic tie-break formula (no formula can know which of two equally-specific keywords is "correct" -- only the person who designed the routing entries knows that) -- add a longer, more specific keyword (`"most cancelled reservations"` , 3 words) to the entry that should win.

- **Bug 2 -- confidently wrong math from an incomplete context, even with real schema info.** `adr_vs_cost` 's context included `mart_aggregates` (revenue/available_nights/occupied_nights) and `dim_accommodation` (total_cost) but not `mart_kpi` (which already has a validated `adr` column). Without it, the model computed "ADR" as `revenue / available_nights` -- that's RevPAN's formula, not ADR's (`revenue / occupied_nights`) -- producing a syntactically perfect, semantically wrong query that returned near-zero values for every accommodation. **Fix**: add `mart_kpi` to `secondary_tables` so the model uses the already-validated `adr` column instead of re-deriving it from raw aggregates.
- **The rule, and the deeper point the user raised while diagnosing this**: good, scoped context measurably reduces hallucination (chapter 07's zero-context run got 6/6 wrong; this chapter's runs got real, verifiable right answers on most questions) -- but it is not a guarantee, and being *too* rigid about which tables a context includes can backfire by forcing the model to reinvent something that already exists correctly elsewhere. The fix for both bugs was the same shape: give the routing config *one more well-chosen piece of information* (a keyword, a table), not a more complex validation mechanism. Chapters 07-13 collectively show a spectrum of defenses -- schema context, routing, DB-level scope, read-only checks -- and this chapter's two live bugs prove none of them, alone or combined, make the system bug-free. They lower the rate and change the *shape* of failures; they don't eliminate the need to actually run the thing against real questions and check the real numbers.

19. A fixed bug that came back on a later run, and what that says about "fixed"

Q: After adding the "most cancelled reservations" keyword and `mart_kpi` , is the cancellation-grain bug (bug 1 above) closed for good?

- **Weak answer**: "Yes, the routing now sends the right context with the right notes."
- **The gap**: while reproducing bug 2's exact SQL for the blog series (post 5), the *same* cancellation-ranking question was re-run with the fully-fixed routing config -- correct `question_type` matched, correct notes present, explicitly warning against using `mart_aggregates` for the ranking. It still picked the wrong month, joining against `mart_aggregates.cancelled_count` (per-accommodation-per-month grain) without aggregating across accommodations first -- the exact same mistake as bug 1, on a config that had already "fixed" a different instance of it. A separate run of the identical question, identical context, got it right.
- **Correct answer**: an LLM is not a deterministic function of (prompt, context) -- the same inputs can produce different outputs across calls. "Fixed" for a routing/documentation change means *the average case improved and a known trigger is gone*, not *this exact mistake can never happen again*. Good context and good notes lower the probability of a given failure; they don't reduce it to zero, and no amount of prompt engineering changes that fact about how the underlying model works.
- **Study**: this is why every number in this project's transcripts was checked against a direct manual query rather than accepted on "the query ran and looks reasonable." A verification step that runs once, during development, catches the bugs that are always present. A verification step that has to run on every real answer is the only thing that catches a bug that's only sometimes present.

20. Robustness is relative to the questions you actually asked

Q: The fixed routing config got 6/6 right on chapter 07's original questions. Does that mean it's robust?

- **Weak answer**: "Yes, 6/6 is a clean result."

- **The gap**: 6/6 on one fixed question set says the config handles *those exact 6 questions*, nothing more. Running 6 new, unremarkable questions -- not adversarial, not edge cases, just different phrasing of reasonable business questions -- surfaced four more real bugs immediately, on the same day, with the exact same config.
- **What the 6 new questions found**: two genuine, real coverage gaps (`avg_length_of_stay_nights` and `avg_booking_window_days` -- both real, validated `mart_kpi` columns -- had never had a routing entry); a false-positive SQL extraction (a model's honest, correct refusal contained the word "with" in ordinary prose, which the `WITH` -accepting regex misread as the start of a CTE, producing a Postgres syntax error out of what was actually a correct plain-English answer); a broken `CASE` comparing against string literals (`'High Season'` / `'Low Season'`) that don't match the real column's actual values (`'HIGH'` / `'LOW'`), which happened to still print the right answer by coincidence, plus an independent window-function misuse that squared a reservation count (1,610 real vs. 2,592,100 returned); and the most serious one -- a routing keyword requiring the literal word "and" (`"occupancy rules and cancellations"`) failed to match a naturally-phrased version of the same question, which fell through to an unrelated type, and the model **silently answered a different, answerable question** (correlating two KPI columns) instead of refusing or flagging the mismatch.
- **The rule**: "robust" is not a property a system has in the abstract -- it's a property relative to a specific set of inputs that have actually been tried against it. A config, a prompt, a validation layer can look completely solid against the test set it was built and tuned against, and still have gaps that only the next slightly-different real question will find. Testing with new, ordinary questions -- not contrived edge cases -- is itself a meaningful robustness check, precisely because it's the kind of input a real system will actually receive.
- **Study**: the most dangerous of the four is not the one that errors (the `WITH` extraction bug) or the one that's obviously a coincidence once you look at the SQL (the season `CASE`) -- it's the one that returns a confident, syntactically valid, seemingly on-topic answer to a question that was quietly substituted for the real one. An honest fallback is a safe failure. A model choosing to answer an adjacent question instead of the one asked, without saying so, is not -- and nothing about the response itself signals that it happened.